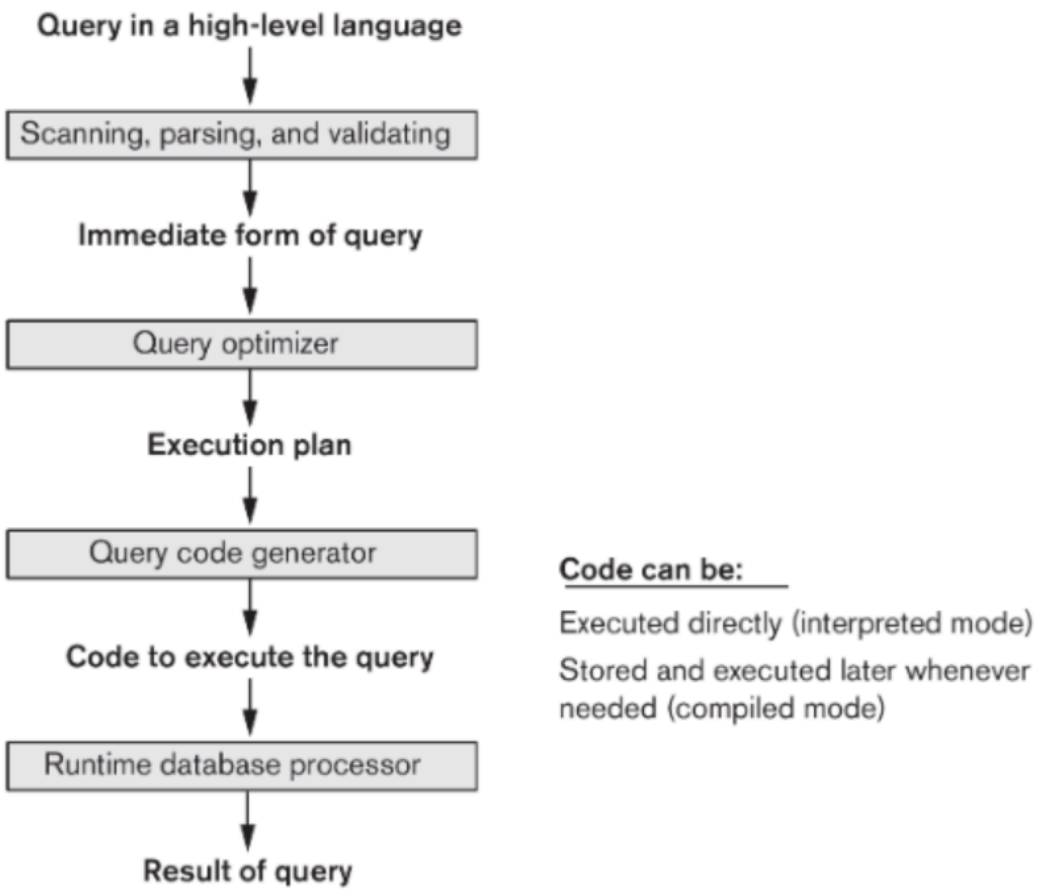
DB15

Query optimization

> **Query를 처리하기 위한 적절한 Execution Plan을 선택하는 과정**

SQL이 Declative language이기 때문에 하나의 Qeury를 처리하는 여러 Execution plan이 존재한다.

과정



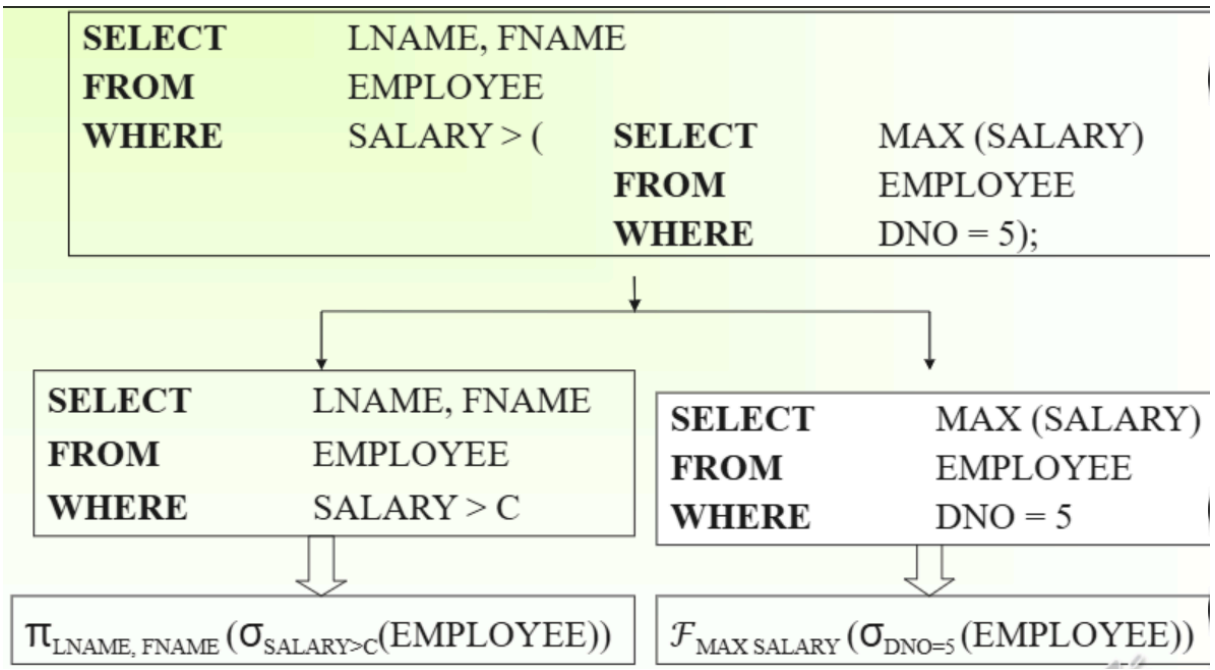
- High-level language에서 **처리해야 할 것이 무엇인지 인식하고 Query가 적절한지 확인**한다.
    - **Immediate form**이라는 의미 위주의 표현을 생성한다.
- **Query optimizer**가 **가장 적절한 Execution plan을 생성**하고 **대응되는 코드는 Query code generator가 생성**한다.
- Query code를 **Runtime database processor가 실제 데이터 위에서 실행한다.**

Query block

> **Relational Algebra로 변경할 수 있고 최적화 할 수 있는 Processing의 기본 단위**

- **SELECT FROM WHERE GROUP BY HAVING** 을 포함한다.
- ORDER BY는 출력 시 관여하는 부분이라 제외한다.

Nested query는 여러 개의 Query block으로 나눌 수 있다.



External Sorting

> **Main memory 밖에서 수행하는 Sorting**

- DB의 경우, 대부분의 경우에 Data가 많기 때문에 External sorting을 이용한다.

Query에 `ORDER BY` , `DISTINCT` 가 포함된 경우에 Sorting이 필요하다.

- 중복 Tuple을 제거하는데 Sorting이 가장 좋다.

Sort-Merge Strategy

1. 전체 File을 **Runs**라고 부르는 **Main memory 크기의 작은 File**들로 나눈다.
2. **한 조각씩 Main memory**에 올려 **Internal Softing**한다.
3. **(2)에서 Sorting한 각 조각 간의 Storing을 추가로 수행**한다.
    - Merge

Disk Access를 줄이는 것이 목적이다.

- Runs를 Main memory에 올리고 정렬된 후 Disk에 다시 적는 IO만 발생한다.
- **Sort-Merge Strategy를 통해 Disk access를 줄일 수 있다.**

Internal sort

Quick sort, Merge sort, Heap sort 등을 이용하여 $O(NlogN)$의 시간 복잡도로 정렬할 수 있다.

Algorithm for processing relational algebra

`Record file` : 실제 Tuple들이 저장된 파일

`Index file` : 검색을 빠르게 하기 위한 보조 파일

> `Primary index` : Record file이 어떤 **key attribute 기준**으로 물리적으로 정렬되어 있을 때 그 attribute 위에 만든 index

`Clustering index` : Record file이 어떤 **Non-key attribute 기준**으로 물리적으로 정렬되어 있을 때 그 attribute 위에 만든 index

`Secondary index` : **Record file의 물리적 정렬과 무관하게** 임의의 attribute로 검색을 빠르게 하기 위해 만든 index file

1. SELECT

S1 Linear search (Brute force, 하나의 결과를 반환)

> **파일의 전체 Record를 검색하며 조건을 만족하는지 확인한다.**

- $O(N)$

S2 Binary search (하나의 결과를 반환)

> **찾고자 하는 Attribute가 이미 Value에 대해 Sorting이 되어 있고 조건이 Key attribute에 대한 '='인 경우에 Binary search를 이용할 수 있다.**

- $O(logN)$

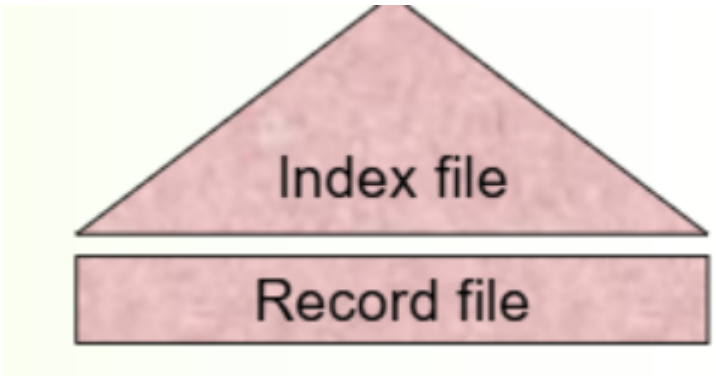
S3 Using Index of Hash key to retrieve single record (하나의 결과를 반환)

> **찾고자 하는 Attribute가 Primary index (Hash)를 가지며 '=' 조건이 Key attribute를 포함하는 경우에 사용하는 방법**

- **B+ Tree**: O(logN)
- **Hash**: O(1)

S4 Primary index를 사용 (여러 결과를 반환)

> **찾고자 하는 Attribute가 Primary inedx를 가지며 조건이 <, ≤, >, ≥ 인 경우에 사용하는 방법**



S3에서 Primary index를 사용할 때와 동일하고 비교 연산자만 다른 것이다.

찾은 곳에서 Subsequent records를 찾는다.

S5 Clustering index를 사용 (여러 결과를 반환)

> **찾고자 하는 Attribute가 Key attribute가 아니며 Clustering index를 갖고 조건이 '='인 경우에 사용할 수 있는 방법**

- Non-key attribute는 중복된 Tuple을 가질 수 있기 때문에 여러 결과를 반환할 수 있다.

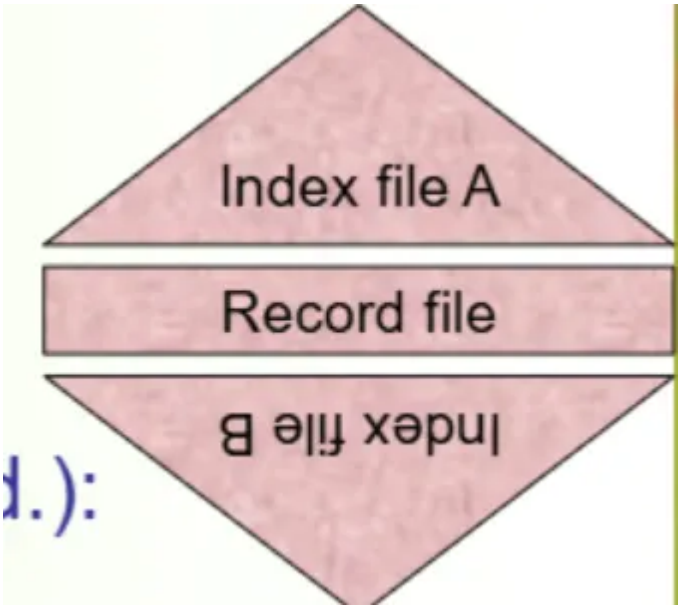
S7 Conjunctive selection

> **Condtion이 Conjunctive한(A AND B AND C ...) 경우에 사용하는 방법**

1. 여러 조건 중 가장 빠른 방법을 사용할 수 있는 조건을 통해 **S2 ~ S5의 방법 중 하나로 후보 Record**를 얻는다.
    - **S2 ~ S5**까지의 방법을 고려하여 가장 빠르게 처리할 수 있는 조건을 선택한다.
    - **여러 Attribute 중 하나 이상이 Sorted거나 Index가 있다면 그 방법대로 수행**
2. 나머지 다른 조건을 이용하여 각 후보 Record를 검사한다.

S9 Conjunctive selection by intersection of record pointers

> **Secondary index file을 사용할 수 있고 Condtion이 A AND B AND C ...인 경우에 두 Condition에 대응되는 Attribute마다 Index가 존재하는 경우에 사용하는 방법**



1. 각 Condition에 대응되는 Index file을 통해 **Record pointer**를 얻는다.
2. (1)에서 얻은 Record pointer의 교집합을 찾는다.
3. (2)에서 얻은 교집합을 후보 Record로 하여 나머지 Condition을 통해 검사한다.

2. JOIN

`Two-way join` : 두 파일 간의 Join

`Multi-way join` : 세 개 이상 파일 간의 Join

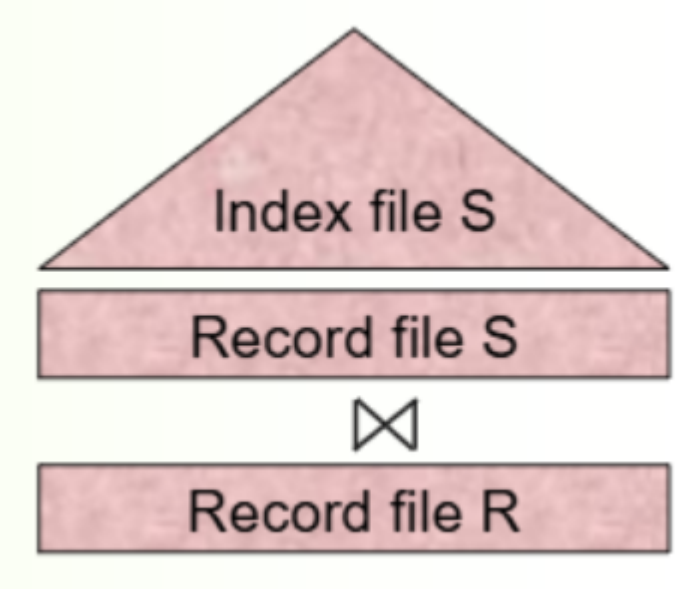
R과 S 사이의 **Equal Join**인 경우의 알고리즘

J1 Nested-loop Join (Brute force)

> **R의 각 Record를 기준으로(Outer loop) S의 모든 Record를 확인하며(Inner loop) Join condition을 만족하는 것이 있는지 확인한다.**

J2 Single-loop Join

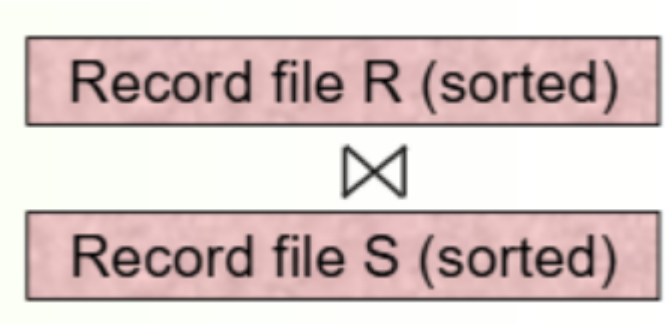
> **두 Relation S에 Join attribute에 대한 Index file (Hash key)이 존재할 때, Index가 존재하지 않는 Relation R에 대해 Loop를 돌며 S에서 각 Record 값과 일치하는 것을 Index로 찾는 방법**



J3 Sort-merge Join

> **두 Record 모두 물리적으로 Join attribute에 대해 정렬되어 있을 때, Join attribute에 대해 같은 값을 가지도록 두 Record file을 scan한다.**

- Sorting이 되어있지 않으면 Sorting을 먼저 한다.
- 각 Record file을 한 번만 Scan한다.



- Sorting부터 해야 한다면 $O(NlogN) + O(NlogN) + O(N)$
- Sorting 되어 있다면 $O(N)$

J4 Hash Join

> **두 Join attribute이 같은 Tuple이 항상 같은 Hash bucket으로 가도록 하는 Hash function을 사용할 때, 둘 중에 더 작은 Relation에 Hash function을 적용하여 Hash bucket을 만들어 놓고 나머지 Record file을 Hash function을 통해 Mapping되는 Hash bucket에서 Join 여부를 확인하는 방법**

- R이 Hash bucket으로 나누어져 있다.
  - `Hash bucket`: Hash function을 적용하였을 때 같은 값의 집합

3. PROJECT

$\pi_{<\text{attribute list}>}(R)$

- **Attribute list에 Key가 포함되지 않으면 Relational algebra이기에 중복 Tuple까지 제거**해야 한다.
  - **Sorting이나 Hashing**을 이용하여 제거할 수 있다.
    - Sorting: 정렬 후 인접한 값에서 같은 Tuple이 존재하는지 확인
    - Hashing: Hash function으로 Hash bucket에 넣고 Bucket 내에서 같은 Tuple이 존재하는지 확인

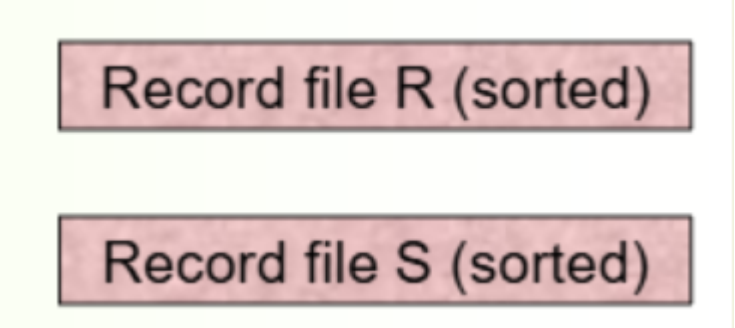
4. SET

UNION, SET DIFFERENCE, INTERSECTION, CARTESIAN PRODUCT

CARTESIAN PRODUCT는 R이 n개의 Record, j개의 attribute를 갖고 있고 S가 m개의 Record, k개의 attribute를 가지고 있을 때 **n*m개의 Record, j+k개의 attribute**를 갖도록 한다.

- **CARTESIAN PRODUCT는 비용이 너무 커서 Query optimizer가 보통 선택하지 않는다.**

두 Record file이 같은 Attribute에 대해 모두 정렬되어 있을 때의 SET Opeartion의 처리는 다음과 같다.



UNION

> **두 Record file을 동시에 Scan하며 Result에 Tuple을 넣는다. 두 Record에 같은 Tuple이 존재한다면 Result에 하나의 Tuple만 넣는다.**

INTERSECTION

> **두 Record file을 동시에 Scan하며 같은 Tuple이 존재하는 경우에만 Result에 하나의 Tuple만 넣는다.**

SET DIFFERENCE (R-S)

> **두 Record file을 동시에 Scan하며 S에 존재하지 않는 R의 Tuple을 Result에 추가한다.**

5. Aggregate operation

Relational algebra는 아니지만, Query block(SQL)에서 사용되기 때문에 필요하다.

MAX, MIN, SUM, AVG, COUNT

Aggregate operation을 구현하기 위해 DBMS는 아래 두 가지 중 하나를 선택한다.

- **Table scan**
- **Index**

MAX

> **(Ascending) Index가 존재한다면 Right-most index를 찾기만 하면 된다.**

- **B-tree를 예시로 들면 Leaf node의 가장 오른쪽 Node를 찾으면 된다.**

MIN

> **(Ascending) Index가 존재한다면 Left-most index를 찾기만 하면 된다.**

- **B-tree를 예시로 들면 Leaf node의 가장 왼쪽 Node를 찾으면 된다.**

결국 MAX / MIN은 Index가 존재하도록 하는 것이 가장 좋다.

SUM, COUNT, AVG

Index의 종류에 따라 구현 방법이 달라진다.

Dense index: 각 Record마다 대응되는 Index가 존재하는 것

- **Index leaf를 모두 읽으며 계산**하면 된다.

Non-dense index: Block (Record의 집합)마다 대응되는 Index가 존재하는 것

- Block 내의 몇 개의 Record가 있는지 등의 정보가 있는 경우도 있다.
  - 이 경우, 이 정보를 이용하면 된다.
- **각 Index에 대응되는 Record의 개수를 고려하여 계산**해야한다.

GROUP BY

- 이 경우에는 Index를 사용하기 어렵다.
- **Grouping attribute에 Sorting이나 Hashing을 사용하여 Group으로 분**할한다.
- 위에서 나눈 **Group에 대해 Aggregate function을 적용**한다.

Using Heuristics in Query Optimization

Heuristic Query Optimization

과정

1. **High-level query parser**가 **Initial internal representation**을 생성한다.
   - 가장 간단하게 Relational algebra가 있다.
2. Internal representation을 최적화 하기 위해 **Heuristics rule**을 적용한다.
   - Algebra 순서를 바꾸는 등 효율적으로 변경.
3. **Query execution plan이 생성**된다.

가장 중요한 Heuristics 원칙

> **Intermidiate results의 size를 줄이는 Operation을 먼저 실행해라**

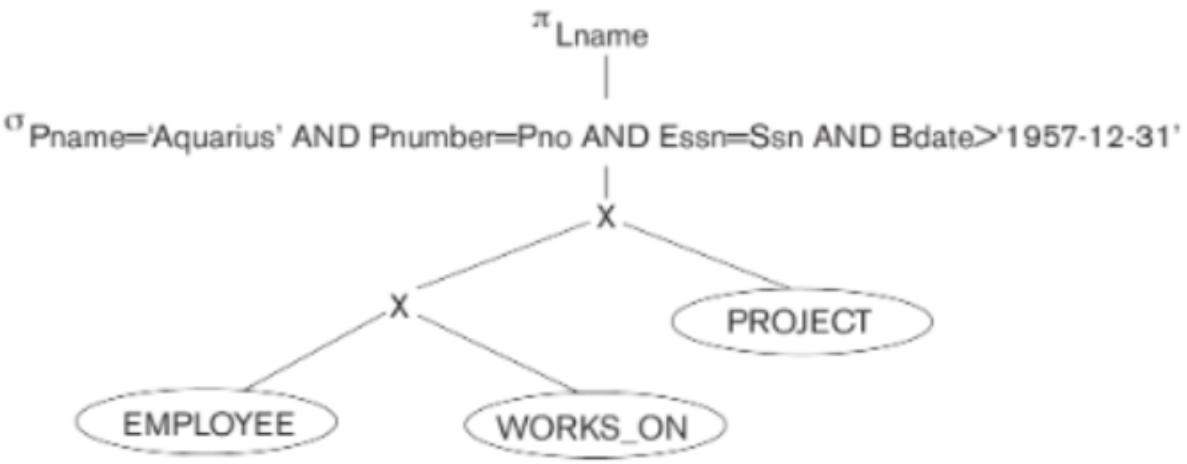
- JOIN 이 전에 SELECT / PROJECT가 먼저 실행되도록 해라.

General transformation rule을 이용해 Query tree를 더 효율적인 형태로 바꾸는 과정.

Query tree

> **Relational algebra expression에 대응되는 Tree date 구조**

- **Leaf node**: Input relation
- **Intermidate node:** Relational algebra operation



- Query tree에서 **중간 연산은 반드시 그것의 Operand가 준비된 후에 가능**하다.
  - Precedure
- 중간에 Relational algebra operation이 Result relation으로 교체되는 것이다.

SQL은 Declarative language인 반면, Relational algebra는 Precedure language이다.

Example1

For every project located in ‘Stafford’, retrieve the project number, the controlling department number, and the department manager’s last name, address, and birthdate

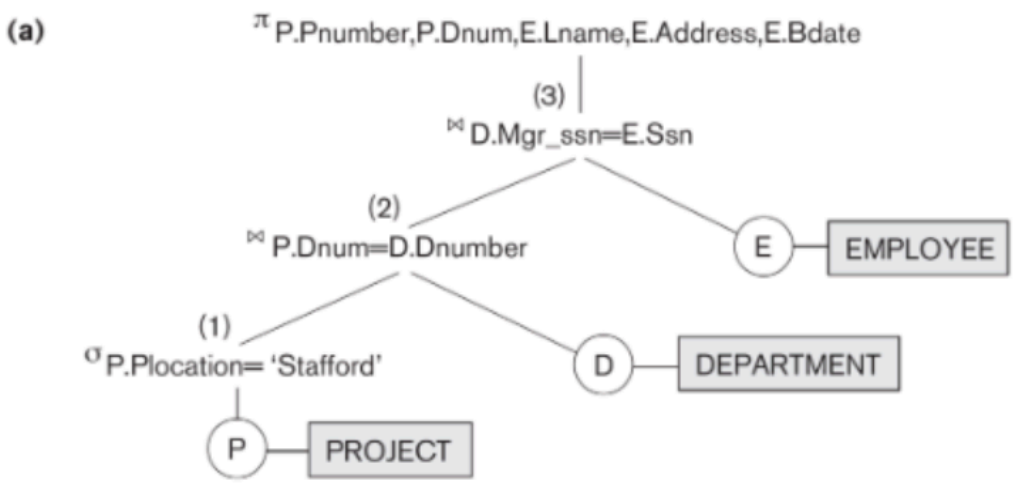
위 요청 사항에 대해 아래와 같은 **SQL query**를 작성할 수 있다.

```
SELECT    P.NUMBER,P.DNUM,E.LNAME,
          E.ADDRESS, E.BDATE
FROM      PROJECT AS P,DEPARTMENT AS D,
          EMPLOYEE AS E
WHERE     P.DNUM=D.DNUMBER AND
          D.MGRSSN=E.SSN AND
          P.PLOCATION='STAFFORD';
```

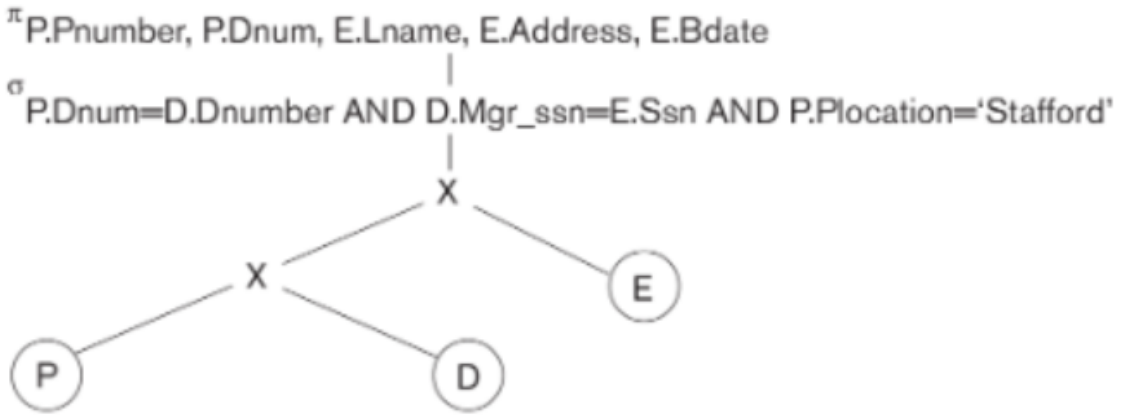
Query optimizer가 **SQL query를 Relational algebra로 변경**한다.

$$\pi_{\text{PNUMBER, DNUM, LNAME, ADDRESS, BDATE}} (((\sigma_{\text{PLOCATION='STAFFORD'}}(\text{PROJECT})) \bowtie_{\text{DNUM=DNUMBER}} (\text{DEPARTMENT})) \bowtie_{\text{MGRSSN=SSN}} (\text{EMPLOYEE}))$$

위 Relational algebra에 대응되는 **JOIN을 이용한 Query tree**는 다음과 같다.



JOIN = CARTESION + SELECT이기 때문에 **CARTESIAN만 이용한 Query tree**는 다음과 같다.



- CARTESIAN PRODUCT의 비용이 크기 때문에 비교적 비효율적이다.

Query graph로 나타낼 수 있다.

- Declarative하다.

하나의 **SQL qeury에 대응되는 Relational algebra expression이 여러 개이기 때문에, 가능한 Query tree 또한 여러 개**이다.

이 중, **가장 효율적인 Query tree**를 찾을 수 있도록 하는 것이 **Heuristic optimization**이다.

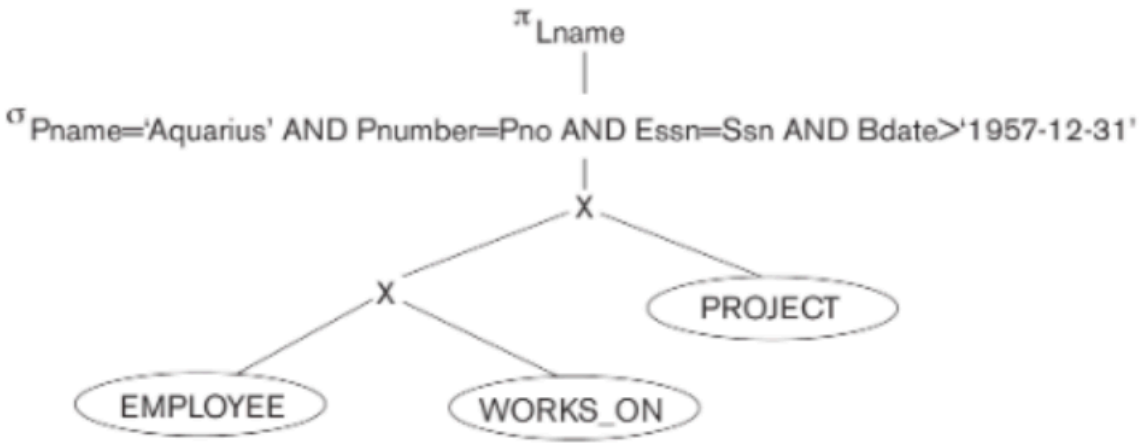
Example2

SQL query는 아래와 같다.

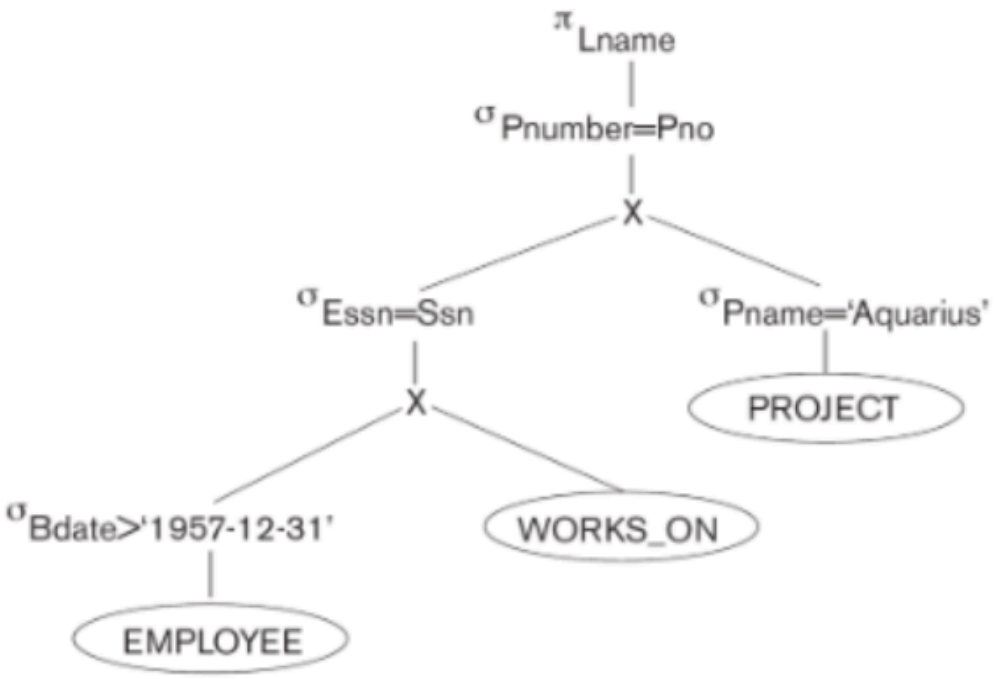
```
SELECT    LNAME
FROM      EMPLOYEE, WORKS_ON, PROJECT
WHERE     PNAME = 'AQUARIUS' AND
          PNMUBER=PNO AND ESSN=SSN
          AND BDATE > '1957-12-31';
```

가장 쉽게 얻을 수 있는 **Query tree는 CARTESIAN PRODUCT를 이용**하는 것이다.

- Initial canonical tree **라고 한다.**



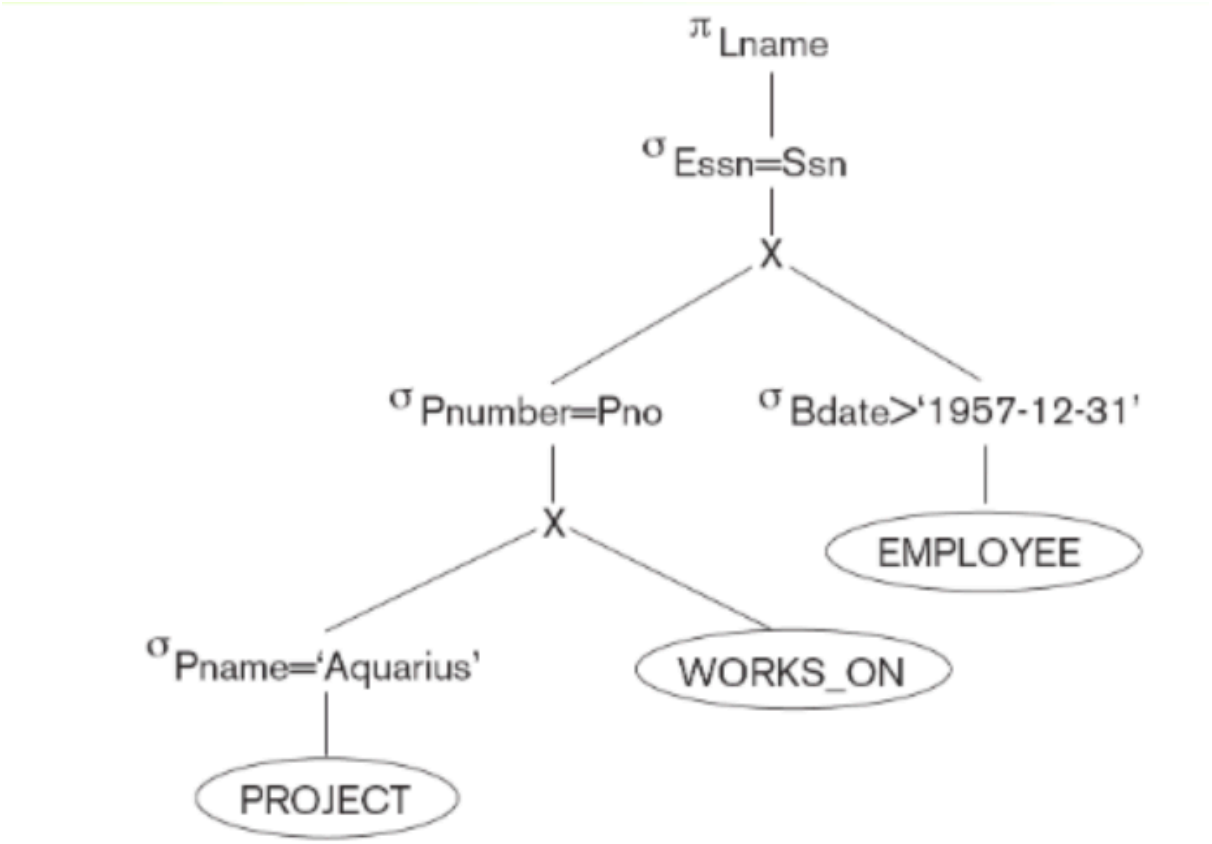
Heuristic optimization의 원칙에 따르면 Relation size를 줄이는 SELECT, PROJECT 연산이 먼저 실행되면 좋다. **SELECT opeation을 아래로 내려보자.**



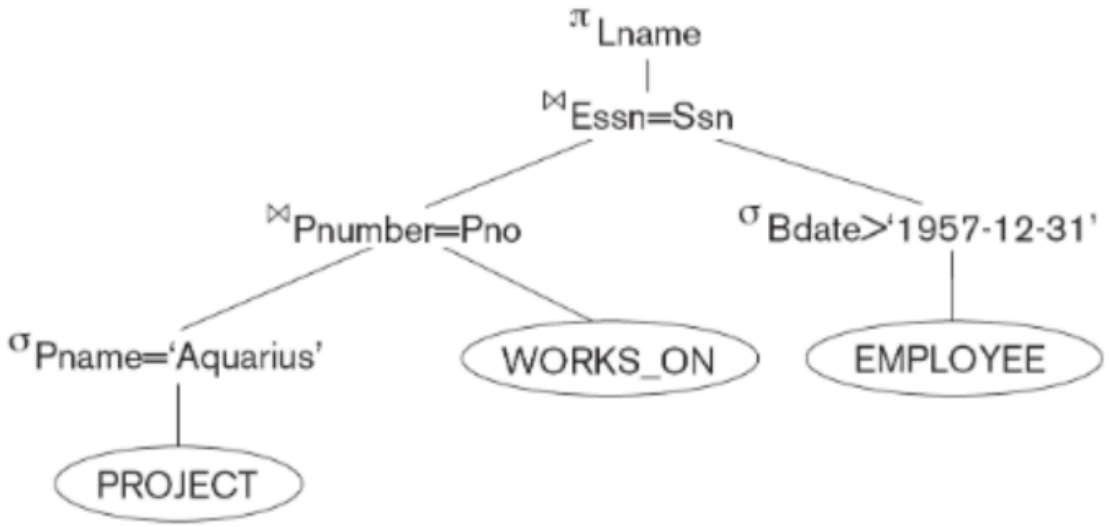
- CARTESIAN PRODUCT를 하더라도 SELECT 후의 작은 Table로 하도록 한다.

More restrictive SELECT를 먼저하도록 하면 더 효율적이다.

- More restrictive SELECT : SELECT를 적용했을 때 **더 많은 tuple을 제거해서 결과 relation의 크기를 더 많이 줄이는** SELECT 조건
- PROJECT에 대한 SELECT가 더 Restrictive하다고 가정하자.

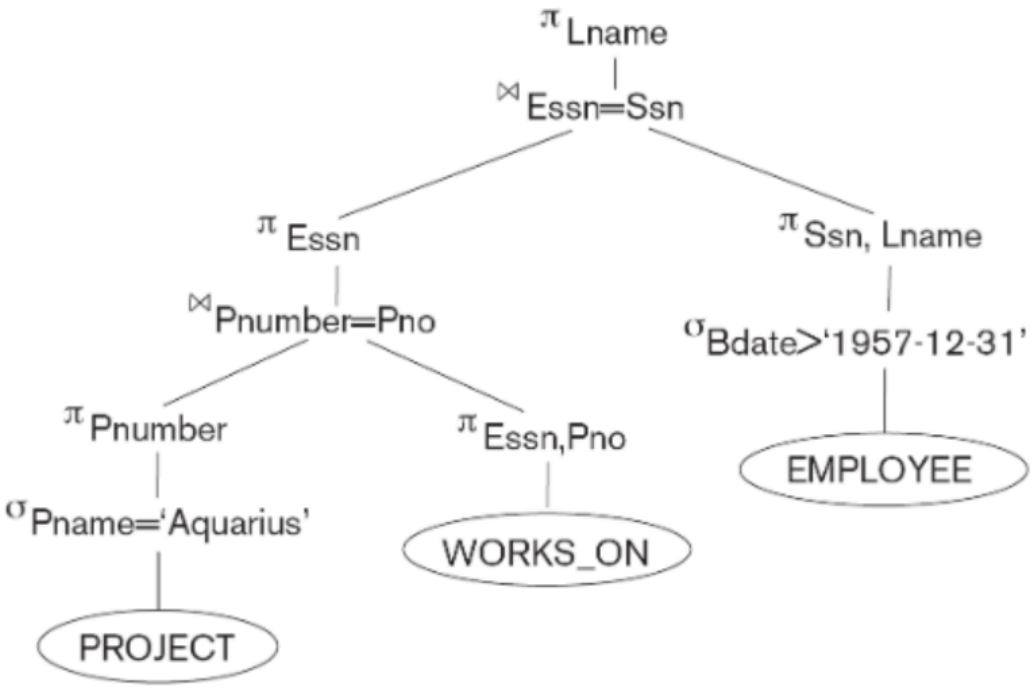


이제, **CARTESIAN + SELECT**를 하나의 JOIN으로 변경할 수 있다.



마지막으로 **PROJECT를 중간마다 추가**하여 Table size를 줄일 수 있다.

- 사용되는 Attribute만 남기는 단계이다.



General Transformation Rules

> **Query의 결과는 바꾸지 않으며 relational algebra expression을 다른 형태로 변환할 수 있게 해주는 규칙**

1. **Cascade of σ**

$$\sigma_{c_1 \text{ AND } c_2 \text{ AND } \ldots \text{ AND } \ldots c_n} = \sigma_{c_1}(\sigma_{c_2}(\ldots(\sigma_{c_n}(R))\ldots))$$

   - 한 번에 여러 조건을 확인하는 것보다 **이전 조건을 확인한 더 작은 Table에 대해 조건을 확인하는 것이 좋다.**
   - **more restrictive한 조건을 먼저 적용할 수 있다.**

2. **Commutativity of σ**

$$\sigma_{c1}(\sigma_{c2}(R)) = \sigma_{c2}(\sigma_{c1}(R))$$

- **c_1에 대한 Index file이 존재하는 경우에 더 좋다.**
- **c_1에 대한 SELECT가 Restirct**한 경우에도 좋다.

3. **Cascade of π**

$$\pi_{List1}(\pi_{List2}(\ldots(\pi_{Listn}(R))\ldots)) = \pi_{List1}(R)$$

- **마지막 PROJECT만 반영**된다.
- 불필요한 연산을 제거한다.

4. **Commuting σ with π**

$$\pi_{A1,A2,\ldots,An}(\sigma_c(R)) = \sigma_c(\pi_{A1,A2,\ldots,An}(R))$$

- c가 $A1, A2, \ldots, An$에 포함된 경우에만 가능하다.

5. **Commutativity of $\bowtie$ (and $\times$)**

$$R \bowtie_c S = S \bowtie_c R; \quad R \times S = S \times R$$

- JOIN에서 **두 Relation을 Outer, Inner로 둘 때, Outer가 큰 것이 Disk access 관점에서 유리**하다.
  - **Inner relation이 작으면 Main memory에 올려놓을 수 있다.**
    - DISK Access 한번으로 Outer의 각 Tuple을 모두 처리할 수 있다.
- 앞에 있는 것이 Outer.

6. **Commuting σ with $\bowtie$ (or $\times$)**

경우 1: 선택 조건 c가 하나의 Relation(예: R)의 속성만 포함할 때

$$\sigma_c(R \bowtie S) = (\sigma_c(R)) \bowtie S$$

경우 2: 선택 조건 c가 $c1 \text{ AND } c2$로 나뉘며, 각각 R과 S의 속성만 포함할 때

$$\sigma_c(R \bowtie S) = (\sigma_{c1}(R)) \bowtie (\sigma_{c2}(S))$$

- SELECT로 Table size를 줄이고 JOIN하는 것이 유리하다.

7. **Commuting π with $\bowtie$ (or $\times$)**

   PROJECT List $L = A_1, \ldots, A_n, B_1, \ldots, B_m$이고, **JOIN Condition c가 L의 속성들만 포함**할 때 성립

$$\pi_L(R \bowtie_c S) = (\pi_{A_1,\ldots,A_n}(R)) \bowtie_c (\pi_{B_1,\ldots,B_m}(S))$$

   - JOIN Condition에 L에 없는 Attribute가 존재한다면, **해당 Attribute를 PROJECT list에 추가한 PROECT → JOIN → 해당 Attribute를 제거한 PROJECT로 사용**할 수 있다.
   - PROJECT로 Table size를 줄이고 JOIN하는 것이 유리하다.

8. **Commutativity of set operations**

$$R \cup S = S \cup R; \quad R \cap S = S \cap R$$

   - SET DIFFERENCE는 성립하지 않는다.
   - 둘 중 **작은 쪽을 Memory**에 올리는 등의 결정을 하기 위함

9. **Associativity of $\bowtie, \times, \cup, \cap$**

$$(R\theta S)\theta T = R\theta(S\theta T)$$

   - 중간 결과 크기를 최소화할 수 있다.

10. **Commuting σ with set operations ($\cup, \cap, -$)**

$$\sigma_c(R\theta S) = (\sigma_c(R))\theta(\sigma_c(S))$$

   - SELECT로 Table size를 줄이고 SET opetaion을 하는 것이 이득이다.

11. **Commuting π with $\cup$**

$$\pi_L(R \cup S) = (\pi_L(R)) \cup (\pi_L(S))$$

- PROJECT로 Table size를 줄이고 UNION하는 것이 이득이다.

12. **Converting a $(\sigma, \times)$ sequence into $\bowtie$**

    CARTESIAN PRODUCT 뒤에 SELECT는 JOIN으로 바꿀 수 있다.

$$(\sigma_c(R \times S)) = (R \bowtie_c S)$$

Using Selectivity and Cost estimate

Cost-based query optimization

> **각기 다른 Execution strategy에 대해 Cost를 추정하고 비교하여 가장 Lowest cost인 Execution strategy를 선택하는 법**

Heuristic query optimization이랑 다르다!

- 규칙 vs 비용
- DBMS는 둘 중 하나만 사용한다.

문제점

1. **Cost function을 정의해야 한다.**
    - 각 Query execution plan에 대한 Cost를 예측하는 함수
    - 정확해야 한다.
2. **Execution strategies의 수가 너무 많아 오래 걸린다.**

Cost component

1. **Access cost to secondary storage**
    - Disk로부터 읽는데 걸리는 시간
2. **Storage cost**
    - Disk에 중간 결과를 쓰는 Cost
3. **Computation cost**
    - Main memory에서의 Join, select 등의 CPU cost
4. **Memory usage cost**
    - Main memory 사용
5. **Communication cost**
    - 여러 Machine을 사용

1번과 2번이 **DISK IO**에 관련되어 가장 중요한 부분이다.

Database system이 처한 환경에 따라 Cost component가 달라진다.

- Secondary memory의 종류 등...